Submission Specification — Redrob Hackathon v4

Read this carefully before submitting. Submissions that don't match this spec will be auto-rejected by the validator without scoring.

---

1. What you're submitting

A CSV file ranking the top **100 candidates** from candidates.jsonl for the released job description.

Rank 1 is the best fit; rank 100 is the 100th best fit.

You do *not* rank candidates 101 onward — only the top 100.

---

2. File format

Filename

Your team's registered participant ID, with .csv extension. For example: team_xxx.csv.

Encoding

UTF-8.

Required columns (in this order)

```
candidate_id,rank,score,reasoning
```

Column	Type	Required?	Description
candidate_id	string	✅ Yes	The CAND_XXXXXXX ID from candidates.jsonl
rank	int (1-100)	✅ Yes	The rank position. Must use each integer 1 through 100 exactly once.
score	float	✅ Yes	Your model's score for this candidate. Should be **monotonically non-**

			increasing as rank increases.
reasoning	string	⚠ Optional but **strongly recommended**	A 1-2 sentence justification explaining why this candidate is at this rank. Used at Stage 4 (manual review) to evaluate top submissions.

Example

```
candidate_id,rank,score,reasoning
CAND_0042871,1,0.987,"Senior AI Engineer with 7 years building RAG systems at product
companies; strong recent engagement and Bangalore-based."
CAND_0019884,2,0.973,"6 years applied ML; previously shipped vector search at scale;
matches the 'product over research' profile in the JD."
CAND_0091235,3,0.962,"Strong NLP + retrieval background; some concern on notice period
(120 days) but otherwise strong fit."
...
CAND_0007729,100,0.412,"Adjacent skills only — likely below cutoff but included as final
filler given experience and engagement signals."
```

---

3. Rules

Format

- **Exactly 100 rows of data** (plus 1 header row).
- Each rank (1 through 100) appears **exactly once**.
- Each candidate_id appears **exactly once**.
- Every candidate_id must exist in the released candidates.jsonl.
- score is non-increasing with rank — i.e., score at rank 1 ≥ score at rank 2 ≥ ... ≥ score at rank 100. Ties are allowed.
- If two candidates have the same score, you must still assign unique ranks. Break score ties deterministically using a secondary signal from your model, or by candidate_id ascending.

Compute constraints

Your code that produces the submission must satisfy the following constraints:

Constraint	**Limit**

Total runtime	≤ 5 minutes wall-clock
Memory	≤ 16 GB RAM
Compute	CPU only — **no GPU** during ranking
Network	Off — your ranking code must not make external API calls (no OpenAI, Anthropic, Cohere, Gemini, or any hosted LLM service)
Disk	≤ 5 GB intermediate state

Why these constraints? This is a real-world recruiting system, not a benchmark. A system that calls GPT-4 or Claude per candidate cannot scale to a 200K candidate pool in production. We want systems that have thought about latency-quality tradeoffs.

In practice, running an LLM call for each of 100,000 candidates will not fit the 5-minute CPU budget, even if the model runs locally. Plan for a small ranker over precomputed features, indexes, or compact local models.

You CANNOT, during the ranking step:

- Call hosted LLM APIs.
- Use GPUs.
- Exceed the runtime/memory limits.

Enforcement. At Stage 3, top-N submissions must provide their full code repository. Your ranking step will be reproduced inside a sandboxed Docker container matching these constraints exactly. **If your submission cannot be reproduced within these limits, it is disqualified at Stage 3**, regardless of your composite score. Make sure your code runs locally on a 16 GB CPU-only machine within 5 minutes before you submit.

Three-submission cap

You may make at most **3 submissions** total during the competition window. Your final entry is your **last valid submission**. Earlier submissions are not preserved.

We've kept this number low intentionally — without a live leaderboard, multiple submissions have limited value, and a low cap reduces gaming.

Reasoning column

The reasoning column is optional but heavily recommended. Top N submissions are advanced to Stage 4 (manual review) where reasoning quality is part of the evaluation.

At Stage 4, we sample 10 random rows from your submission and check each reasoning entry against the following:

Check	What we're looking for
Specific facts	Does the reasoning reference specific facts from the candidate's profile (years of experience, current title, named skills, signal values)?
JD connection	Does the reasoning connect to specific JD requirements, not just generic praise?
Honest concerns	Where the candidate has obvious gaps or concerns, does the reasoning acknowledge them?
No hallucination	Does every claim in the reasoning correspond to something actually in the candidate's profile? Skills, employers, or experience that don't exist in the profile are red flags.
Variation	Are the 10 sampled reasonings substantively different from each other (not templated)?
Rank consistency	Does the reasoning's tone match the rank? A rank-5 candidate with critical reasoning, or a rank-95 candidate with glowing reasoning, indicates the reasoning was generated independently of the ranking.

What's penalized:

- Empty reasoning
- All-identical reasoning strings
- Templated reasoning that just inserts the candidate's name
- Reasoning that mentions skills not in the candidate's profile (hallucination)
- Reasoning that contradicts the rank

Plain-language reasoning that demonstrates you actually understood the candidate's profile will rank highly here. Don't try to be impressive; try to be specific and honest.

---

4. How submissions are scored

Metrics

Your top-100 ranking is scored against the **hidden ground truth** using these metrics:

Metric	Weight	What it measures
NDCG@10	0.50	Quality of your top-10 picks
NDCG@50	0.30	Quality of your top-50 picks
MAP (Mean Avg Precision)	0.15	Precision across all relevance levels
P@10	0.05	Fraction of top-10 that are "relevant" (tier 3+)

Final composite

Final composite = 0.50 × NDCG@10 + 0.30 × NDCG@50 + 0.15 × MAP + 0.05 × P@10

Scoring happens **once**, after submissions close. There is no public partition, no live leaderboard, and no per-submission feedback during the competition. Your score is computed against the full hidden ground truth and is revealed only when final results are announced.

Tiebreaks

If two submissions have identical composites:

1. Higher P@5 wins.
2. Higher P@10 wins.
3. Earlier submission timestamp wins.

---

5. Evaluation pipeline (stages)

Your submission flows through these stages:

Stage	What happens	What gets you eliminated
1. Format validation	Auto-validator runs on every submission	Any spec violation in section 3
2. Scoring	Composite computed once on the full hidden ground truth, after submissions close	Final score below cutoff for advancement to Stage 3
3. Code reproduction + honeypot check	Top-N submissions: full code repo requested. Ranking step reproduced in sandboxed environment (5min, 16GB, no	Cannot reproduce within compute limits; honeypot rate >10% in top 100; missing or fabricated code repo

	GPU, no network). Honeypot rate computed.	
4. Manual review	Reasoning quality (6 checks above). Methodology coherence. Git history authenticity (real iteration vs single dump). Code quality.	Failed reasoning checks; flat git history with no iteration; codebase consists entirely of LLM API calls
5. Defend-your-work interview	Top X finalists: 30-minute video call with Redrob engineering. Walk through architecture, defend design choices, demonstrate familiarity with your own code.	Cannot explain architecture; contradicts submitted code; clearly didn't build it

Note on AI tool usage: You are allowed to use AI tools (Claude, GPT-4, etc.) as part of your development workflow. We expect many participants will. The evaluation is designed so that AI-assisted submissions where the human did real engineering work will succeed, while submissions that are mostly LLM output with minimal human engineering will fail at Stages 3-5. The compute constraint, code repo check, and defend-your-work interview together filter for genuine engineering, not for absence of AI use.

---

6. Common rejections (we see these every hackathon)

- 99 rows or 101 rows instead of exactly 100.
- Ranks starting at 0 instead of 1.
- Duplicate candidate_ids.
- candidate_id typos that don't exist in candidates.jsonl.
- All scores set to the same value (model isn't differentiating).
- Scores increasing as rank increases (rank 1 has lowest score).
- Submission file submitted as .xlsx or .json instead of .csv.

Double-check these locally before uploading — the server-side auto-validator rejects on any of them.

---

7. Honeypot warning

The dataset contains a small number (~80) of **honeypot candidates** with subtly impossible profiles (e.g., 8 years of experience at a company founded 3 years ago; "expert" proficiency in 10 skills with 0 years used). These are forced to relevance tier 0 in the ground truth.

If your submission ranks honeypots in the top 10, this is a strong signal that your system isn't reading profiles — it's just doing keyword embedding. We use the honeypot rate as a Stage 3 filter: submissions with honeypot rate > 10% in top 100 are disqualified.

You can identify honeypots through careful profile inspection. We expect a good ranking system to naturally avoid them; you don't need to special-case them.

---

8. Leaderboard policy

The leaderboard is hidden during the competition. You will not see your score until final results are announced. We strongly recommend you validate your approach locally using methodology and reasoning, not by submitting many variations.

---

9. Sample submission

A sample submission CSV that matches this spec is included in your hackathon bundle as sample_submission.csv. It is **not** a high-quality ranking — it's only a format reference.

---

10. What you submit (full picture)

Your submission consists of three parts, all required:

10.1 The CSV file

The top-100 ranking, as specified in Sections 2 and 3.

10.2 Portal metadata

Collected at upload time via the submission form. Have these ready before you start the upload:

Field	Required?	Notes
Team name	✅ Yes	Used in leaderboard and result announcements
Primary contact name	✅ Yes	One person to act as your team's point of contact
Primary contact email	✅ Yes	Used for all organizer communication

Primary contact phone	☑ Yes	Used for top-N / top-X communication
GitHub repository URL	☑ Yes	Must be reachable. Private repos OK if you can grant access to organizers at Stage 3. Format: https://github.com/USERNAME/REPO
Sandbox / demo link	☑ Yes	A working hosted environment where your ranking system can be run. See Section 10.5 below.
AI tools declared	☑ Yes	Multi-select: Claude / ChatGPT / Copilot / Cursor / Gemini / Other / None. Honest declaration, not penalized.
Compute environment summary	☑ Yes	One line describing where you ran your code (e.g., "MacBook Pro M2, 16GB RAM, Python 3.11")
Team member list	☑ Yes	Name + email for each member
Methodology summary	Optional	≤200 words explaining your approach. Strongly recommended — helps at Stage 4 review.

10.3 Code repository

Your GitHub repo should include:

- A clear README.md with setup instructions and exact commands to reproduce your submission CSV
- The full source code that produced the CSV (no hidden steps, no manual edits)
- Any pre-computed artifacts your code depends on (embeddings, indexes, model weights), or a script that produces them
- A requirements.txt, pyproject.toml, or equivalent specifying all dependencies and versions
- A submission_metadata.yaml at the repo root mirroring your portal metadata (use the template provided in the hackathon bundle as submission_metadata_template.yaml)

For Stage 3 code reproduction, your README must indicate **a single command** that produces the submission CSV from the candidates file. For example:

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

If your system requires pre-computation (e.g., generating embeddings), document this clearly — pre-computation may exceed the 5-minute window, but the **ranking step** that produces the CSV must complete within it.

10.4 AI tools declaration — what it means

The hackathon **permits** AI tool use. We've designed the evaluation pipeline so that AI-assisted submissions where the human did real engineering work will succeed, while AI-only submissions (paste-and-pray) will fail at Stage 3 (compute reproduction), Stage 4 (no real code repo), or Stage 5 (cannot defend the work).

The declaration is for transparency, not filtering. Be honest. If your interview answers contradict your declaration, that's a much stronger negative signal than the AI use itself.

10.5 Sandbox / demo link requirement

A sandbox is a hosted environment where organizers (and you) can verify your ranking system runs reproducibly. Acceptable sandbox platforms include:

- **HuggingFace Spaces** (free tier is fine)
- **Streamlit Cloud** (free tier is fine)
- **Replit** (public repl)
- **Google Colab** (with link to a notebook that runs end-to-end)
- **A docker pull + docker run link** to a public registry image
- **A binder link** for a runnable Jupyter notebook

Your sandbox needs to:

4. Accept a small candidate sample (≤100 candidates) as input — either via upload or pre-loaded
5. Run your ranking system end-to-end and produce a ranked CSV
6. Complete within the compute budget (≤5 min on CPU)

It does **not** need to handle the full 100K pool — small-sample reproducibility is what we're checking. The full reproduction at Stage 3 happens in our own sandbox.

Why it's mandatory: at Stage 3 we will reproduce your full ranking step from your GitHub repo. The sandbox is a faster, lower-stakes sanity check that lets us (and you) verify the code runs at all before we invest in full reproduction. Submissions without a working sandbox link are flagged at Stage 1.

If you have a strong reason a hosted sandbox is impractical for your approach, you can submit a self-contained docker run recipe in your GitHub README instead — but the dockerfile must build and run unmodified.